

Lab 3. Các câu lệnh DML (Data Manipulation Language) trong MySQL

1. SQL Select

SELECT dùng để lấy dữ liệu từ một table (nhiều table) trong một (hoặc nhiều) database.

Cú pháp:

```
Select col1, col2 from tb1;
```

Trong đây:

Tb1: là tên bảng tb1;

Col1 và col2 là tên hai cột trong bảng tb1.

Ví dụ: trong bảng SinhVien của database CSDL_Thuchanh; lấy ra 2 trường dữ liệu HoSinhVien và TenSinhVien;

```
use csdl_thuchanh;
```

```
select HoSinhVien, TenSinhVien from Sinhvien;
```

Kết quả trả về:

	HoSinhVien	TenSinhVien
►	Nguyễn Văn	Quang
	Tran Ngoc	Han
	Ha Duy	Lap
	Tran Ngoc	Linh
	Tran Minh	Long
	Le Nhat	Minh

Chạy câu lệnh sau, xem kết quả là gì:

```
Select HoSinhVien as Ho, TenSinhVien as Ten from Sinhvien;
```

Để lấy tất cả trường dữ liệu trong một bảng ta dùng câu lệnh sau:

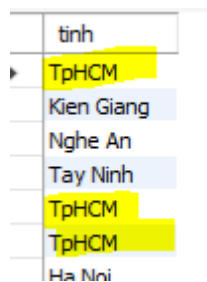
```
Select * from Table_name;
```

Ví dụ: select * from Sinhvien;

2. SQL Select DISTINCT

Chúng ta sử dụng **Distinct** để loại bỏ các giá trị giống nhau. Ví dụ như Table SinhVien ở trên nếu ta viết câu lệnh lấy thuộc tính *Tinh*:

```
Use CSDL_Thuchanh;  
Select Tinh from SinhVien;
```

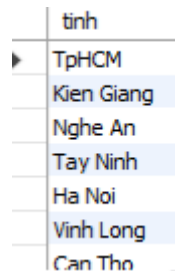


tinh
TpHCM
Kien Giang
Nghe An
Tay Ninh
TpHCM
TpHCM
Ha Noi

Dễ thấy các giá trị thuộc tính: TpHCM là trùng lặp.

Giờ chúng ta dùng từ khóa **Distinct** để loại bỏ chúng:

```
use csdl_thuchanh;  
select distinct tinh from sinhvien;;
```



tinh
TpHCM
Kien Giang
Nghe An
Tay Ninh
Ha Noi
Vinh Long
Can Tho

Các giá trị thuộc tính lặp lại đã bị loại bỏ, mỗi giá trị thuộc tính chỉ xuất hiện một lần duy nhất trong bảng kết quả trả về.

Xét tiếp câu lệnh tiếp theo, kết quả trả về là gì. Tại sao lại vậy?

```
use csdl_thuchanh;  
select distinct tinh, gioitinh from sinhvien;
```

3. SQL Where

Chúng ta sử dụng mệnh đề Where khi muốn lọc lại các kết quả. Chúng ta thêm điều kiện muốn lọc sau từ khoá Where.

Cấu trúc câu lệnh như sau:

```
SELECT col1, col2  
From TenBang  
WHERE dieukien;
```

Ví dụ 1: Lấy các thuộc tính của bảng city với giá trị (cột) city là Hanoi;

```
USE Sakila  
SELECT * FROM sakila.city  
Where city = 'hanoi';
```

3.1 Các loại toán tử có thể sử dụng trong mệnh đề WHERE là

Toán tử so sánh

Toán tử	Mô tả
=	Toán bằng
>	Lớn hơn
<	Nhỏ hơn
>=	Lớn hơn hoặc bằng
<=	Nhỏ hơn hoặc bằng
<>	Không bằng

Toán tử logic

Toán tử	Mô tả
And	TRUE nếu tất cả điều kiện là TRUE
BETWEEN	TRUE nếu tất cả nằm trong range
IN	TRUE nếu câu query thoả mãn một trong những điều kiện
LIKE	TRUE nếu thoả mãn pattern đưa ra
NOT	Hiển thị dữ liệu nếu điều kiện không phải True
OR	TRUE nếu 1 trong 2 điều kiện là TRUE

4. SQL And

Chúng ta sử dụng từ khóa And khi muốn kết hợp hai hay nhiều điều kiện. Cấu pháp như sau:

```
SELECT cot1, cot2, cot3,...  
FROM bảng 1, bảng 2  
WHERE (điều kiện1) AND (điều kiện 2)
```

Ví dụ : Hiện thị giá trị từ 2 bảng city và country với quốc gia có mã country_id là 105;

Use sakila
Select * from city, country
where country.country_id = 105 **and** city.country_id = 105;

5. SQL OR

Chúng ta sử dụng OR để kiểm tra một trong hai điều kiện có đúng hay không. Ví dụ ta muốn tìm tất cả dữ liệu của bảng city nếu giá trị cột City là Hanoi hoặc nếu không tìm thấy Hanoi thì City là HaiPhong thì cũng hiển thị kết quả. Chỉ cần một trong hai điều kiện thỏa mãn là được.

Use sakila;
SELECT * FROM sakila.city
Where (city ='hanoi') **SELECT * FROM** sakila.city
Where (city ='hanoi') or (city = 'haiphong'); (city = 'haiphong');

6. SQL NOT

Chúng ta sử dụng NOT để phủ định lại kết quả điều kiện. Ví dụ tìm tất cả khách hàng mà first_name không phải là Mary.

Use sakila;
SELECT * FROM sakila.customer
where not (first_name = 'mary');

7. Kết hợp AND OR NOT

Chúng ta có thể kết hợp **AND OR NOT** trong một câu truy vấn như sau:
Trong bảng city, chọn những thành phố là Hanoi và Vinh, với country id là 105;

SELECT * FROM sakila.city
Where (country_id = 105) **and** ((city ='hanoi') **or** (city = 'Vinh'));

Câu lệnh sẽ ưu tiên thực thi trong dấu () trước sau đó so sánh **And**.

SELECT * FROM sakila.city
Where Not (country_id = 105) **and** Not (country_id = 20);

8. SQL ORDER BY

Chúng ta sử dụng từ khoá ORDER BY khi muốn sắp xếp dữ liệu tăng dần hoặc giảm dần.

```
SELECT cot1, cot2, cot3,...  
FROM bảng 1, bảng 2  
WHERE (điều kiện1) AND (điều kiện 2)  
ORDER BY col1, col2... ASC hoặc DESC;
```

ASC dùng để sắp xếp theo thứ tự tăng dần

DESC dùng để sắp xếp kết quả theo thứ tự giảm dần

Ví dụ

```
SELECT * FROM sakila.city  
Where Not (country_id = 105) and Not (country_id = 20)  
Order by country_id ;
```

```
SELECT * FROM sakila.city  
Where Not (country_id = 105) and Not (country_id = 20)  
Order by city DESC;
```

9. SQL Insert

Chúng ta sử dụng từ khoá Insert để thêm 1 dòng vào Database.

Cú pháp

```
INSERT INTO tên bảng (cột1, cột2,...)
```

```
Values (Value1, value2,...);
```

Ví dụ: Thêm 2 bạn sinh viên vào bảng sinhvien của database CSDL_Thuchanh;

```
Insert into Sinhvien()
```

```
values ('K1307','Nguyen tran','Nam','2003-02-02','nu','','BacGiang',''),
```

```
('K1308','Nguyễn Hồng','Nhưng','2003-02-02','nu','','Hanoi','');
```

10. SQL Update

Chúng ta sử dụng từ khoá **Update** để cập nhật lại giá trị trong Table.

○ Cú pháp

```
UPDATE tên bảng
```

```
SET tên cột = giá trị cần cập nhật
```

```
WHERE điều kiện
```

Ví dụ.

Chúng ta thay đổi cột giới tính của bạn sinh viên có mã sinh viên là K1307 từ *nữ* thành *nam* như sau.

```
SET SQL_SAFE_UPDATES = 0;
```

```
UPDATE Sinhvien
```

```
SET gioitinh = 'nam'
```

```
WHERE masinhvien = 'K1307';
```

```
SET SQL_SAFE_UPDATES =1;
```

11. SQL Delete

Chúng ta sử dụng từ khoá Delete khi muốn xoá một dòng trong Table.

```
DELETE FROM tên bảng WHERE điều kiện xoá;
```

Ví dụ: Xóa bạn sinh viên có mã sinh viên là K1307 khỏi bảng SinhVien:

```
SET SQL_SAFE_UPDATES = 0;
```

```
DELETE from sinhvien
```

```
where MaSinhVien='k1307';
```

```
SET SQL_SAFE_UPDATES =1;
```

Để xóa tất cả các dữ liệu ta làm như sau:

```
Delete FROM SinhVien;
```

Hoặc dùng **truncate;**

Thực hành

Thực hành trong database Sakila;

Bài 1. Sử dụng từ khóa Distinct cho bảng City để lấy ra các country_id

Bài 2. Lấy ra 2 thuộc tính Category_id và Name trong bảng category.

Bài 3. Sử dụng distinct để lấy customer_id và staff_id cho bảng payment.

Bài 4. Làm lại bài 1, 2 và 3, với từ khóa Order by theo thứ tự giảm dần của 1 cột thuộc tính trong bảng

Bài 5. Lấy các dòng dữ liệu có amount lớn hơn 4 trong bảng Payment sau đấy sắp xếp theo thứ tự giảm dần của cột thuộc tính Rental_id.

Bài 6. Lấy các dòng dữ liệu có amount lớn hơn 4 và staff_id là 2 trong bảng Payment sau đấy sắp xếp theo thứ tự giảm dần của cột thuộc tính Rental_id.